The Common Canvas

The projective geometry of how CudaSift maps two images onto one pixel grid

CudaSift 3.0.0 (rct-scientific-proc fork), commit 97708c5. Companion to *Image Stitching in CudaSift*. Every number in this document comes from running the built library on the same synthetic pair used there, and every formula was checked against the library's output in NumPy.

Scope. This document covers one thing in depth: the mathematics by which a 3×3 homography places the pixels of two images onto a single canvas. It begins with homogeneous coordinates and what a homography does to points and lines, introduces the three coordinate frames the code works in and the matrices between them, derives why the footprint of image 2 is a convex quadrilateral and when the four-corner bounding box is valid, explains inverse mapping and bilinear reconstruction including their local geometry (the Jacobian), and ends with a fully traced numerical example and a NumPy reference implementation that reproduces the kernel. Feature detection, matching and the estimation of H are out of scope.

Contents

1. Setting and notation . . . 3
2. Homogeneous coordinates and homographies . . . 3
2.1 Points as rays . . . 3
2.2 What a homography is . . . 4
2.3 Three properties the warp relies on . . . 4
2.4 Where the homography comes from . . . 4
2.5 The inverse by cofactors . . . 5
3. Three frames and the matrices between them . . . 5
4. The footprint of image 2 in the reference frame . . . 6
4.1 Transporting the corners . . . 6
4.2 Why four corners are enough . . . 6
4.3 Straight, but not evenly spaced . . . 7
4.4 The bounding box and the canvas . . . 8
5. Pulling pixels: inverse mapping . . . 8
5.1 Why the warp runs backwards . . . 8
5.2 The per-pixel chain . . . 9
5.3 What happens near the exceptional line . . . 9
5.4 The first output is an exact copy . . . 9
6. Bilinear reconstruction . . . 10
6.1 Definition . . . 10
6.2 The domain, and the lost last row and column . . . 10
6.3 What interpolation does to the signal . . . 11
6.4 Local geometry: the Jacobian . . . 11
7. Validity masks and the overlap . . . 12
8. Worked example, end to end . . . 13
8.1 Canvas . . . 13
8.2 Four canvas pixels traced by hand . . . 13
8.3 The NumPy re-implementation against the kernel . . . 15
9. Numerical remarks . . . 15
10. The mapping chain in one table . . . 16
Appendix A. NumPy reference implementation . . . 17
Appendix B. The example's numbers at a glance . . . 18

1. Setting and notation

Two grayscale images are given, image 1 of size $w_1 \times h_1$ and image 2 of size $w_2 \times h_2$, together with a homography **H** produced by cusift_estimate_homography. The library's convention, fixed by the way the RANSAC residual is written, is that **H** carries image-1 coordinates to image-2 coordinates. The warp stage produces a canvas size outW × outH, an origin (u_0, v_0), and two float images C_1 and C_2 of that size. The size, the origin and the canvas positions of the corners of both images are also available without warping, through `cusift_compute_canvas`. The two canvases are defined pixel by pixel as

$$C_1(x, y) = I_1(x + u_0, y + v_0), \qquad C_2(x, y) = I_2(\varphi(x, y)), \qquad \varphi = \pi \circ \mathbf{H} \circ \mathbf{T}^{-1}$$

where π is the perspective divide, **T** is the integer translation that turns reference coordinates into canvas coordinates, and a value is NaN wherever the argument falls outside the image. Everything in this document unpacks that line.

Symbol	Meaning
(u, v)	coordinates in the reference frame, which is the frame of image 1
(x, y)	canvas pixel indices, 0 ≤ x < outW, 0 ≤ y < outH
(s_x, s_y)	the point of image 2 that a canvas pixel reads
H, $\mathbf{H}^{-1}$	the homography image 1 → image 2, and its inverse
T	reference → canvas: $(u, v) \mapsto (u - u_0, v - v_0)$
π	perspective divide $(x, y, z) \mapsto (x/z, y/z)$
$I(x, y)$	intensity of an image at a real-valued position, by bilinear interpolation of its pixels
$q_0 \dots q_3$	the four corners of image 2 transported into the reference frame
F	the footprint of image 2 in the reference frame, the quadrilateral q0 q1 q2 q3
z	the third homogeneous coordinate after applying H; the denominator of the perspective divide
J	the 2×2 Jacobian of (u, v) → (sx, sy)

Pixel positions. Pixel (i, j) of an image, column i and row j, sits at the point (i, j) of its frame. There is no half-pixel offset anywhere in the library: keypoint coordinates, the homography and the warp all use this convention, so a pixel index and a position are interchangeable. The continuous image $I(x, y)$ is defined by bilinear interpolation on the half-open square $[0, w-1) \times [0, h-1)$; section 6 explains why the upper bound is open.

2. Homogeneous coordinates and homographies

2.1 Points as rays

A point (x, y) of the plane is represented by any non-zero triple $(x, y, 1) \cdot \lambda$ with $\lambda \neq 0$. All multiples describe the same point; what matters is the direction of the triple as a ray through the origin of $\mathbb{R}^3$. The map back to the plane is the perspective divide

$$\pi(x, y, z) = (x / z, y / z), \qquad z \neq 0.$$

Triples with $z = 0$ have no finite image: they are the points at infinity, one for each direction in the plane, and together they form the line at infinity. The representation is useful because a translation, which is not linear on (x, y), becomes linear on the triple: adding (t_x, t_y) is multiplication by a matrix whose third column is $(t_x, t_y, 1)$. This is why the whole warp, translation included, is a chain of matrix products.

2.2 What a homography is

A homography (also projective transformation) is an invertible linear map of the triples, hence an invertible 3×3 matrix, read modulo scale: **H** and λ**H** move every point to the same place because the scale disappears in π. Nine entries minus one for scale leave 8 degrees of freedom, which is why four point correspondences (two equations each) determine one. CudaSift removes the scale ambiguity by storing **H** with $h_{22} = 1$, which the refinement enforces; the warp kernel nevertheless uses h_{22} as given, so a caller-supplied matrix with any non-zero scale also works.

$$\begin{vmatrix} x' \\ y' \\ z \end{vmatrix} = \begin{vmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & h_{22} \end{vmatrix} \begin{vmatrix} u \\ v \\ 1 \end{vmatrix}, \quad s_x = x' / z, \quad s_y = y' / z$$

Written out, the action on plane coordinates is the pair of rational functions

$$s_x = (h_{00}u + h_{01}v + h_{02}) / z, \quad s_y = (h_{10}u + h_{11}v + h_{12}) / z, \quad z = h_{20}u + h_{21}v + h_{22}.$$

The numerators are affine in (u, v); all the non-linearity sits in the division by z. When $h_{20} = h_{21} = 0$ the map is affine (parallel lines stay parallel, 6 degrees of freedom); if in addition the upper-left block is a scaled rotation it is a similarity (4 degrees of freedom), the second model CudaSift can fit.

2.3 Three properties the warp relies on

- **Lines go to lines.** A line is the set of triples with $\ell^T\mathbf{x} = 0$ for some coefficient triple ℓ. If $\mathbf{x}' = \mathbf{H}\mathbf{x}$, then $\ell^T\mathbf{H}^{-1}\mathbf{x}' = 0$, so the image is the line with coefficients $\mathbf{H}^{-T}\ell$. Consequently the image of a line segment is a line segment, provided the segment does not cross the line that **H** sends to infinity (next point). Section 4 uses this to show that the footprint of image 2 is a quadrilateral.
- **One line is sent to infinity.** The points with $z = 0$, that is $h_{20}u + h_{21}v + h_{22} = 0$, have no finite image; their coefficients are the third row of **H**. Points on opposite sides of this line get opposite signs of z. For the example, the third row of **H** is (-4.145e-05, -3.080e-05, 1.000) and the line lies about 19 000 px away from image 1; the third row of $\mathbf{H}^{-1}$ puts its line about 18 000 px from image 2. Both images are far on one side, which is the normal situation for a stitch.
- **Composition is matrix multiplication and the inverse is the matrix inverse.** If **H** takes frame 1 to frame 2 then $\mathbf{H}^{-1}$ takes frame 2 to frame 1, and chaining maps is chaining matrices. The warp composes **H** with a translation; the canvas sizing composes $\mathbf{H}^{-1}$ with nothing but reads four points through it.

2.4 Where the homography comes from

Two pinhole views of the same plane, or two views from a camera that only rotates, are related exactly by a homography. With calibration matrices $\mathbf{K}_1$, $\mathbf{K}_2$, relative rotation **R**, translation **t**, and the plane $\mathbf{n}^T\mathbf{X} = d$ in the first camera's coordinates,

$$\mathbf{x}_2 \simeq \mathbf{K}_2 \left(\mathbf{R} - \mathbf{t}\,\mathbf{n}^T / d\right) \mathbf{K}_1^{-1}\,\mathbf{x}_1, \quad \text{and for pure rotation} \quad \mathbf{x}_2 \simeq \mathbf{K}_2\,\mathbf{R}\,\mathbf{K}_1^{-1}\,\mathbf{x}_1.$$

Neither formula is used by the code; the matrix is fitted directly from matches. The derivation matters for two reasons. It says when a single matrix can align two images at all (planar scene, or no parallax), which is why moving objects and depth relief turn into ghosts. And it explains the size of the entries: for cameras that are nearly parallel the third row is close to (0, 0, 1), so z stays near 1 across the image. In the example the perspective coefficients are of order 10^{-5}, and z ranges only from about 1.03 to 1.12 at the corners of image 2.

2.5 The inverse by cofactors

`Invert3x3` (src/cusift.cu:767) computes $\mathbf{H}^{-1}$ = adj(**H**) / det(**H**) with the adjugate written out entry by entry: for **H** = [*a b c*; *d e f*; *g h i*],

$$\det \mathbf{H} = a(ei - fh) - b(di - fg) + c(dh - eg),$$

$$\mathbf{H}^{-1} = (1/\det) \cdot \begin{vmatrix} ei - fh & ch - bi & bf - ce \\ fg - di & ai - cg & cd - af \\ dh - eg & bg - ah & ae - bd \end{vmatrix}$$

The call fails when $|\det \mathbf{H}| < 10^{-12}$. Because $\det(\lambda\mathbf{H}) = \lambda^3 \det \mathbf{H}$, that threshold is only meaningful with the $h_{22} = 1$ normalisation the library maintains. For a near-similarity with scale σ the determinant is about σ^2, so the guard only catches genuinely degenerate matrices, never a legitimate small scale. In the example det **H** = 0.87705, and the cofactor result agrees with a floating-point LU inverse to machine precision.

3. Three frames and the matrices between them

The code reasons in three coordinate systems. The frame of image 1 is the **reference frame**: nothing is ever resampled in it, and the canvas is simply a window onto it. Image 2 has its own frame, related to the reference by **H**. The **canvas frame** is the reference frame shifted so that the window's top-left pixel is (0, 0).

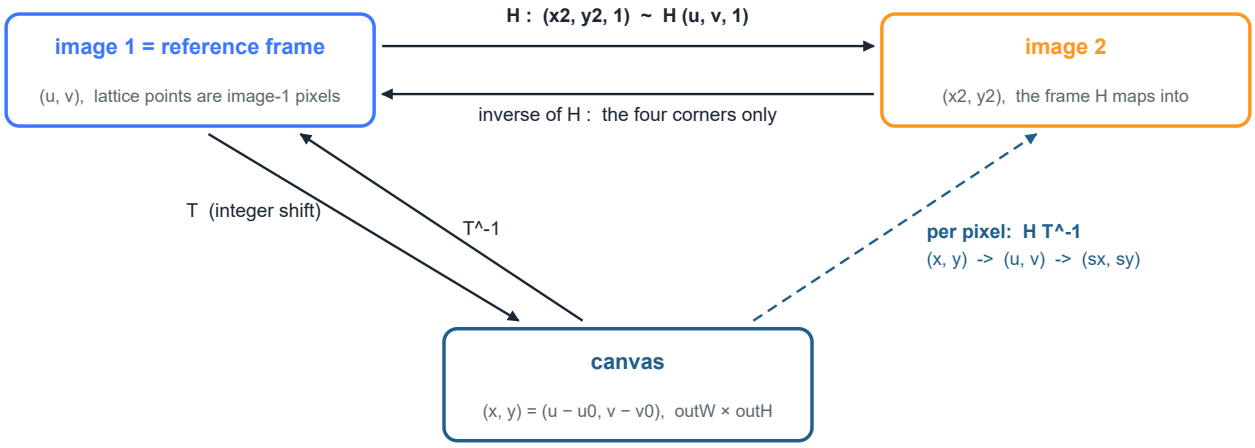


Figure 1. The three frames and the six maps between them. The per-pixel warp uses the dashed composite; the canvas size uses H^{-1} on four points only.

$$\mathbf{T} = \begin{vmatrix} 1 & 0 & -u_0 \\ 0 & 1 & -v_0 \\ 0 & 0 & 1 \end{vmatrix}, \quad \mathbf{T}^{-1} = \begin{vmatrix} 1 & 0 & u_0 \\ 0 & 1 & v_0 \\ 0 & 0 & 1 \end{vmatrix}$$

$$\mathbf{HT}^{-1} = \begin{vmatrix} h_{00} & h_{01} & h_{02} + h_{00}u_0 + h_{01}v_0 \\ h_{10} & h_{11} & h_{12} + h_{10}u_0 + h_{11}v_0 \\ h_{20} & h_{21} & h_{22} + h_{20}u_0 + h_{21}v_0 \end{vmatrix}$$

From	To	Matrix	Where the code uses it
reference (u, v)	image 2	$\mathbf{H}$	per canvas pixel, after the shift
image 2	reference	$\mathbf{H}^{-1}$	the four corners of image 2, to size the canvas
reference	canvas (x, y)	$\mathbf{T}$	implicit: canvas pixel = reference point minus (u0, v0)
canvas	reference	$\mathbf{T}^{-1}$	u = x + originU, v = y + originV in the kernel
canvas	image 2	$\mathbf{HT}^{-1}$	the whole per-pixel map; the kernel applies the two factors in turn
image 2	canvas	$\mathbf{TH}^{-1}$	never applied to pixels; it is what the footprint figure shows

T is special: u_0 and v_0 are integers, so **T** maps the integer lattice $\mathbb{Z}^2$ of the reference frame onto the integer lattice of the canvas. Canvas pixels *are* image-1 pixel positions, shifted by a whole number of columns and rows. Section 5 shows that this makes the first output an exact copy, and section 6 shows why a half-pixel shift would not be harmless.

4. The footprint of image 2 in the reference frame

4.1 Transporting the corners

The footprint F is the set of reference-frame points that image 2 covers: F = { $\pi(\mathbf{H}^{-1}\,\mathbf{p})$: **p** in the rectangle $[0, w_2-1] \times [0, h_2-1]$ }. The code transports only the four corners:

$$q_k = \pi\,(\,\mathbf{H}^{-1}\,\mathbf{c}_k\,), \quad \mathbf{c}_k \in \{\,(0,0,1),\,(w_2-1,\,0,\,1),\,(w_2-1,\,h_2-1,\,1),\,(0,\,h_2-1,\,1)\,\}$$

corner of image 2	coordinates	z through $\mathbf{H}^{-1}$	position in the reference frame
q_0	(0, 0)	1.03186	(759.026, -19.057)
q_1	(w2−1, 0) = (1149, 0)	1.08593	(1875.082, 45.601)
q_2	(w2−1, h2−1) = (1149, 999)	1.11836	(1826.295, 978.557)
q_3	(0, h2−1) = (0, 999)	1.06430	(741.774, 963.264)

4.2 Why four corners are enough

The rectangle is the convex hull of its corners, and the question is whether its image under a projective map is the convex hull of the four transported corners. In general it is not: a projective map can tear a region apart if the region crosses the line that is sent to infinity. The precise statement is the following.

Claim. Let ℓ be the line that $\mathbf{H}^{-1}$ sends to infinity (its coefficients are the third row of $\mathbf{H}^{-1}$). If the rectangle of image 2 lies entirely on one side of ℓ, then z has one sign at every point of the rectangle, $\pi \circ \mathbf{H}^{-1}$ is a continuous bijection of the rectangle onto its image, every edge maps onto the straight segment between the images of its endpoints, and the image is the convex quadrilateral with vertices q_0, q_1, q_2, q_3.

Why. On one side of ℓ the denominator never vanishes, so the map is continuous there. A projective map preserves lines (section 2.3) and, more strongly, preserves the order of points along a line that does not meet the exceptional line, so an edge, which is a segment, maps to a segment with the same endpoints. Convexity of the image follows: a segment between two image points is the image of the segment between their preimages, which lies in the rectangle. The condition is easy to verify numerically: the four values of z must share a sign. In the example they are all positive, between 1.032 and 1.118, and ℓ is roughly 18 000 px from the nearest corner. If the condition failed, the bounding box computed from the corners would be wrong and the true footprint unbounded; in practice such an **H** comes from a failed estimate, and the size guard of section 4.4 usually rejects it.

4.3 Straight, but not evenly spaced

Figure 2 draws the pixel lattice of image 2, every 100 pixels, transported into the reference frame. The lines are straight, as the claim predicts, but they are no longer parallel and their spacing drifts across the image. That drift is the perspective component: a homography preserves lines and the cross-ratio of four collinear points, but not ratios of lengths. The midpoint of the top edge of image 2 lands at fraction 0.5128 of the transported edge, not 0.5. The local scale therefore varies continuously, and section 6.4 quantifies it with the Jacobian.

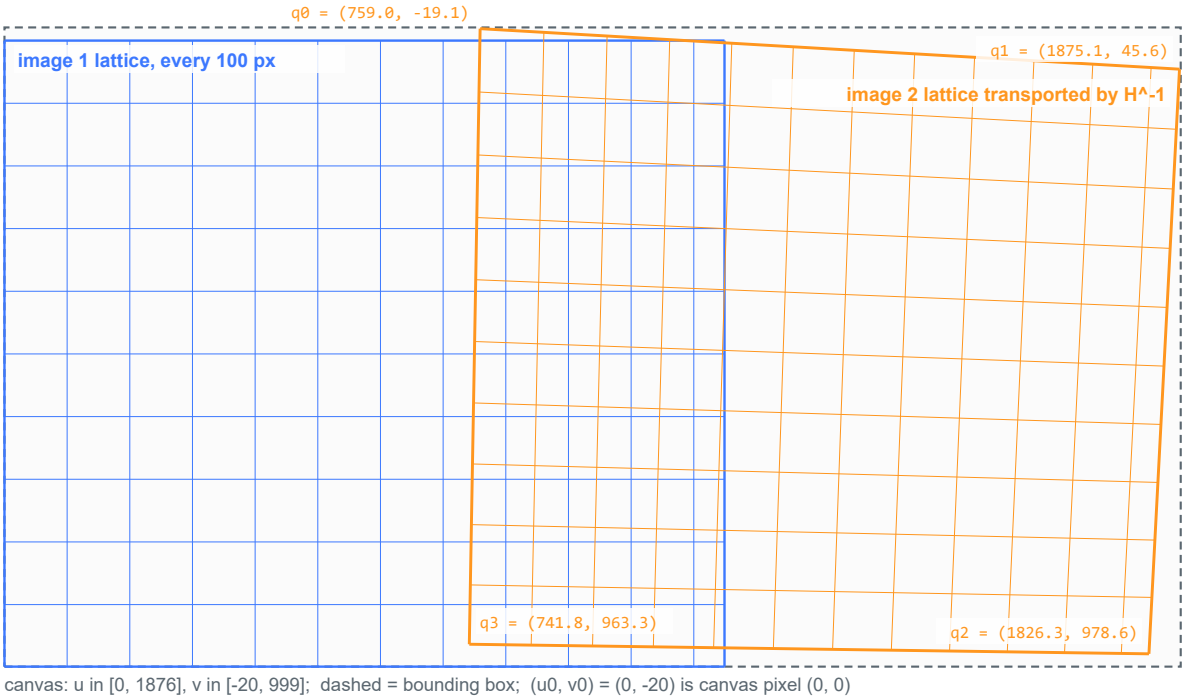


Figure 2. The lattice of image 2 (orange, every 100 px) transported into the reference frame through H^{-1}, over the lattice of image 1 (blue). Lines stay straight; spacing and angles do not. The dashed rectangle is the canvas.

4.4 The bounding box and the canvas

The canvas must hold image 1 unchanged and all of F. Both are convex, so the smallest axis-aligned rectangle containing them is determined by extreme coordinates:

$$u_{\min} = \min(0, \min_k q_{k,x}), \quad u_{\max} = \max(w_1 - 1, \max_k q_{k,x}), \quad \text{and likewise } v_{\min}, v_{\max} \text{ with } h_1 - 1.$$

$$u_0 = \lfloor u_{\min} \rfloor, \quad u_1 = \lceil u_{\max} \rceil, \quad v_0 = \lfloor v_{\min} \rfloor, \quad v_1 = \lceil v_{\max} \rceil, \quad \text{outW} = u_1 - u_0 + 1, \quad \text{outH} = v_1 - v_0 + 1.$$

- **Rounding outward** (floor on the minimum, ceiling on the maximum) guarantees that every reference point covered by either image has a canvas pixel at or beyond it in each direction; no part of F is clipped.
- **The +1** makes the range inclusive: the canvas holds the integer columns u_0, u_0+1, …, u_1, which are $u_1 - u_0 + 1$ of them.
- **$u_0 \le 0$ and $v_0 \le 0$ always**, because the minimum is taken together with 0, the origin of image 1; likewise $u_1 \ge w_1 - 1$. The canvas never crops image 1. It can only grow: when F lies entirely inside image 1, outW × outH = $w_1 \times h_1$. This is what happens for the bundled test pair, where H is nearly the identity.
- **The example:** $u_{\min} = \min(0, 741.8) = 0$ and $u_{\max} = 1875.1$, so $u_0 = 0$ and $u_1 = 1876$; $v_{\min} = -19.1$ gives $v_0 = -20$, and $v_{\max} = 999$ gives $v_1 = 999$. The canvas is 1877 × 1020 with origin (0, -20): canvas pixel (0, 0) is reference point (0, -20), which belongs to neither image.

The size guard. Before allocating, the code requires outW ≤ 2·max(w_1, w_2) and outH ≤ 2·max(h_1, h_2). For a similarity with scale σ (image-2 coordinates = σ × reference coordinates) the footprint has width about $(w_2-1)/\sigma$, so a scale below roughly one half, or a translation of more than about one image width, trips the guard. It is deliberately loose: its job is to refuse the absurd canvases that come from a wrong **H**, and it gives `cusift_memory_warp` a bound to budget against. Area is a sharper check on a correct **H**: the shoelace area of F is 1,054,998 px², against 1,147,851 px² for image 2 itself, a ratio of 0.919 ≈ $1/\sigma^2$ for σ ≈ 1.043, consistent with **H** magnifying by about that factor.

Where this lives in the code. Sections 4.1 to 4.4 are implemented once, in the internal helper `ComputeCanvasGeometry` (src/cusift.cu:805), which every warp entry point calls before allocating. The public `cusift_compute_canvas` (src/cusift.cu:1155) runs the same helper on its own and returns a `cusift_canvas`: the canvas size, the origin (u_0, v_0), and the canvas coordinates of the four corners of each image, in constant time and without touching the GPU. Everything drawn in Figure 2 can therefore be obtained from one call, and it matches the canvas a subsequent warp allocates exactly, because the two share the code.

5. Pulling pixels: inverse mapping

5.1 Why the warp runs backwards

The natural first idea, push every pixel of image 2 through $\mathbf{TH}^{-1}$ and drop it on the canvas, does not work on a lattice. The images of lattice points are not lattice points, so each would have to be spread over its neighbours; where the map stretches, some canvas pixels receive nothing and show holes; where it compresses, several sources pile into one pixel and must be normalised. The cure is to turn the question around: for each canvas pixel ask which point of the source it corresponds to, and read the source there.

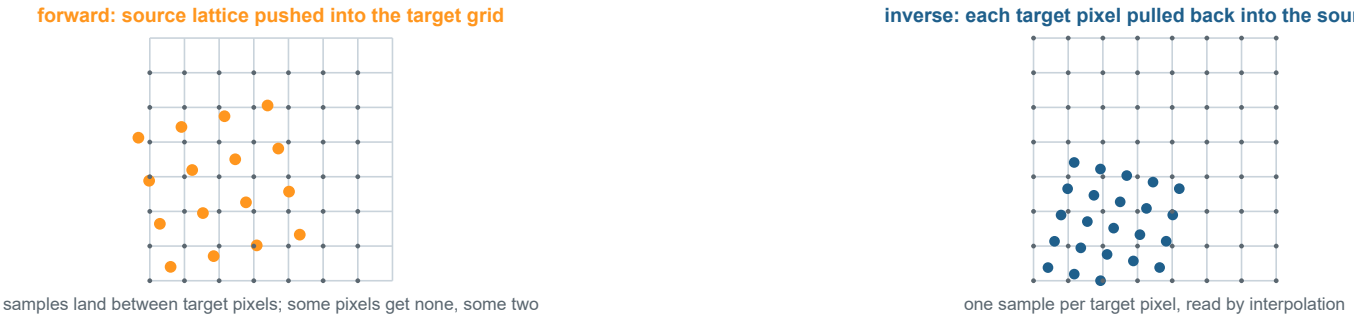


Figure 3. Forward mapping of a source lattice (left) scatters samples irregularly over the target grid. Inverse mapping (right) visits every target pixel exactly once and reads the source at a real-valued position.

Formally the output is the composition $C_2 = I_2 \circ \varphi$ with $\varphi = \pi \circ \mathbf{H} \circ \mathbf{T}^{-1}$. Writing it this way makes two things explicit: the only matrix applied per pixel is **H** (the inverse is never used inside the kernel), and the only operation that touches pixel values is the evaluation of I_2 at a non-integer point, which is interpolation. The warp is therefore exactly as good as its interpolation; the geometry is exact.

5.2 The per-pixel chain

`WarpDualKernel` (src/cusift.cu:911) runs one thread per canvas pixel and evaluates, in this order,

$$u = x + u_0, \quad v = y + v_0 \quad \text{(this is } \mathbf{T}^{-1}\text{)}$$

$$z = h_{20}u + h_{21}v + h_{22}, \quad s_x = (h_{00}u + h_{01}v + h_{02}) / z, \quad s_y = (h_{10}u + h_{11}v + h_{12}) / z \quad \text{(}\mathbf{H}\text{, then } \pi\text{)}$$

$$C_2(x, y) = \text{bilinear}(I_2; s_x, s_y) \text{ if } 0 \le s_x < w_2 - 1 \text{ and } 0 \le s_y < h_2 - 1, \text{ else NaN.}$$

Adding (u_0, v_0) first and multiplying by **H** afterwards is the same as multiplying by the composite $\mathbf{H}\mathbf{T}^{-1}$ of section 3, whose third column absorbs the shift. Pre-composing would save two additions per pixel at the cost of a different rounding pattern; the kernel keeps the two steps separate, which also keeps (u, v) available for the first output.

5.3 What happens near the exceptional line

The kernel does not test z. If a canvas pixel lay on the line $z = 0$, the division would give $\pm\infty$ or NaN; on the far side z is negative and the point is "behind" the plane at infinity. In every such case the range test fails, because a comparison with NaN is false and $\pm\infty$ violates one of the two bounds, so the output is NaN rather than a read out of bounds. The design is safe by construction; it is the canvas size, not the kernel, that assumes the exceptional line stays away from the images.

5.4 The first output is an exact copy

For image 1 the chain stops after $\mathbf{T}^{-1}$: the kernel samples I_1 at (u, v) with the same bilinear routine. But $(u, v) = (x + u_0, y + v_0)$ is an integer pair, so the fractional parts are exactly zero and the interpolation collapses to the single pixel value. Image 1 is never blurred, and the first canvas is a pure translation of image 1 with NaN padding. The NumPy re-implementation of Appendix A reproduces the library's first canvas bit for bit (exactly: maximum difference 0).

6. Bilinear reconstruction

6.1 Definition

Let (s_x, s_y) be the real-valued sample position, $x_0 = \lfloor s_x \rfloor$, $y_0 = \lfloor s_y \rfloor$, and $f_x = s_x - x_0$, $f_y = s_y - y_0$ the fractional parts in [0, 1). Bilinear interpolation reads the four lattice neighbours and combines them with weights that are products of the one-dimensional fractions:

$$I(s_x, s_y) = (1-f_x)(1-f_y)\, I_{00} + f_x(1-f_y)\, I_{10} + (1-f_x)f_y\, I_{01} + f_x f_y\, I_{11}, \quad I_{ij} = I(x_0+i, y_0+j).$$

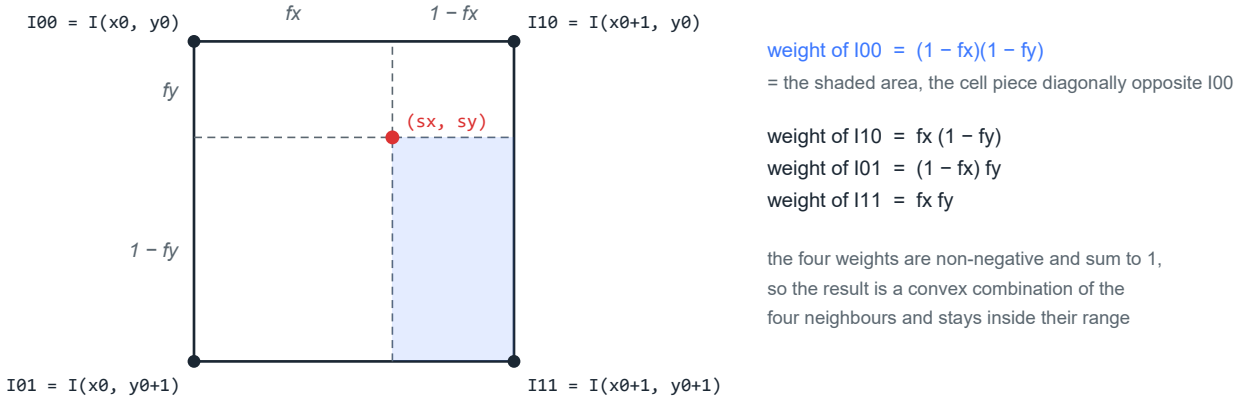


Figure 4. The unit cell around a sample. Each neighbour's weight is the area of the sub-rectangle diagonally opposite it, so the nearest neighbour gets the largest weight and the weights sum to one.

Three equivalent readings are useful. It is two linear interpolations, first along x on the two rows, then along y between the results (or the other order; the result is the same). It is the unique function that is bilinear, that is, affine in each coordinate separately, on every lattice cell and matches the pixel values at the lattice points. And it is a convolution: with the triangle (hat) function $\Lambda(t) = \max(0, 1 - |t|)$,

$$I(s_x, s_y) = \sum_{m,n} I(m, n)\, \Lambda(s_x - m)\, \Lambda(s_y - n),$$

because only the four neighbours have a non-zero hat value. The hats form a partition of unity, so the four weights are non-negative and sum to one, the result is a convex combination that never leaves the range of the four neighbours, and the interpolant is continuous across cell boundaries (but not differentiable: it has creases along the lattice lines).

6.2 The domain, and the lost last row and column

The formula needs $x_0+1 \le w-1$, that is $s_x < w-1$, and likewise in y. The kernel's test is exactly that: $0 \le s_x < w-1$ and $0 \le s_y < h-1$. The domain of the continuous image is therefore the half-open square $[0, w-1) \times [0, h-1)$, and a sample exactly on the last column or row is rejected even though the formula would degenerate gracefully there. The consequence is visible in the counts: the first canvas has 1,147,851 valid pixels, which is $(1150-1)(1000-1) = 1{,}147{,}851$ exactly, one column and one row fewer than image 1.

A related detail: the C cast `(int)sx` truncates toward zero, which equals the floor only for non-negative arguments. The lower bound of the test guarantees that, so $f_x, f_y \in [0, 1)$ always holds and the weights are never negative.

6.3 What interpolation does to the signal

Sampling between lattice points is a low-pass operation. In one dimension, reading at fractional offset f is the filter with taps $(1-f, f)$, whose frequency response has magnitude $|1 - f + f\cdot e^{-i\omega}|$. At the Nyquist frequency $\omega = \pi$ this is $|1 - 2f|$, which is 1 at the lattice ($f = 0$) and 0 at the half-pixel offset ($f = 0.5$): the finest alternating pattern an image can hold is erased completely when sampled midway between pixels. Averaged over all offsets, bilinear interpolation attenuates the top octave substantially while leaving low frequencies untouched. This is the cost paid by the second canvas and avoided by the first, and it is the reason the integer origin of section 3 is worth having: a non-integer origin would blur image 1 for no benefit.

6.4 Local geometry: the Jacobian

How many image-2 pixels does one canvas pixel represent? Differentiate the rational map of section 2.2. With $N_x = h_{00}u + h_{01}v + h_{02}$ and $s_x = N_x/z$, the quotient rule gives $\partial s_x/\partial u = (h_{00}z - N_x h_{20})/z^2 = (h_{00} - s_x h_{20})/z$, and the same pattern for the other three entries:

$$\mathbf{J} = (1 / z) \cdot \begin{vmatrix} h_{00} - s_x h_{20} & h_{01} - s_x h_{21} \\ h_{10} - s_y h_{20} & h_{11} - s_y h_{21} \end{vmatrix}$$

$|\det \mathbf{J}|$ is the local area ratio, image-2 pixels per canvas pixel; the singular values of $\mathbf{J}$ are the stretch factors along the two principal directions. For an affine map the Jacobian is constant; for a projective map it varies with position through z and through the sample coordinates themselves. Two global checks tie the local and global pictures together. The analytic Jacobian agrees with central finite differences of the map to 10^{-6}. And by the change of variables formula the integral of $|\det \mathbf{J}|$ over F equals the area of image 2's domain:

$$\int_F |\det \mathbf{J}|\, du\, dv = (w_2-1)(h_2-1) = 1{,}147{,}851$$ versus the lattice sum $\sum |\det \mathbf{J}|$ over the 1,055,003 valid canvas pixels = 1,147,856.

Reference point (u, v)	z	det J	singular values	reading
overlap centre (950, 500)	0.9452	1.0385	1.0290, 1.0092	minified: more than one source pixel per canvas pixel
corner (0,0) of image 1	1.0000	0.8770	0.9723, 0.9021	magnified: less than one source pixel per canvas pixel
corner (w1-1,0)	0.9524	1.0153	1.0226, 0.9929	minified: more than one source pixel per canvas pixel
corner (w1-1,h1-1)	0.9216	1.1204	1.0736, 1.0436	minified: more than one source pixel per canvas pixel
corner (0,h1-1)	0.9692	0.9632	1.0192, 0.9451	magnified: less than one source pixel per canvas pixel
canvas right edge, mid height (1876, 500)	0.9068	1.1761	1.1021, 1.0671	minified: more than one source pixel per canvas pixel

Across the footprint det $\mathbf{J}$ runs from 0.964 to 1.227: image 2 is slightly minified on the right and slightly magnified on the left, as the converging lattice in Figure 2 shows. Bilinear reconstruction reads four neighbours regardless of det $\mathbf{J}$. Where det $\mathbf{J}$ is well above 1, source pixels fall between the samples and are skipped, which aliases fine texture; CudaSift applies no pre-filter or area averaging, so strong

minification (a stitch across a large zoom) is the one situation where its warp is noticeably worse than a resampling library's. For near-unit scale, the normal stitching case, it is adequate.

7. Validity masks and the overlap

Each canvas carries its own validity, encoded as not-NaN. In set terms,

$$V_1 = \{\,(x, y) : (x+u_0, y+v_0) \in [0, w_1-1) \times [0, h_1-1)\,\}, \quad V_2 = \{\,(x, y) : \varphi(x, y) \in [0, w_2-1) \times [0, h_2-1)\,\}.$$

V_1 is a rectangle of lattice points. V_2 is the set of lattice points inside the preimage of the half-open square, which is the quadrilateral F of section 4 with two of its edges removed. The overlap $V_1 \cap V_2$ is the lattice part of the intersection of a rectangle and a convex quadrilateral, itself a convex polygon with at most eight sides. Clipping F against the rectangle of image 1 (Sutherland–Hodgman) gives a polygon of area 385,179 px²; the kernel reports 384,852 overlapping pixels. The small difference is the half-open boundary and the lattice itself.

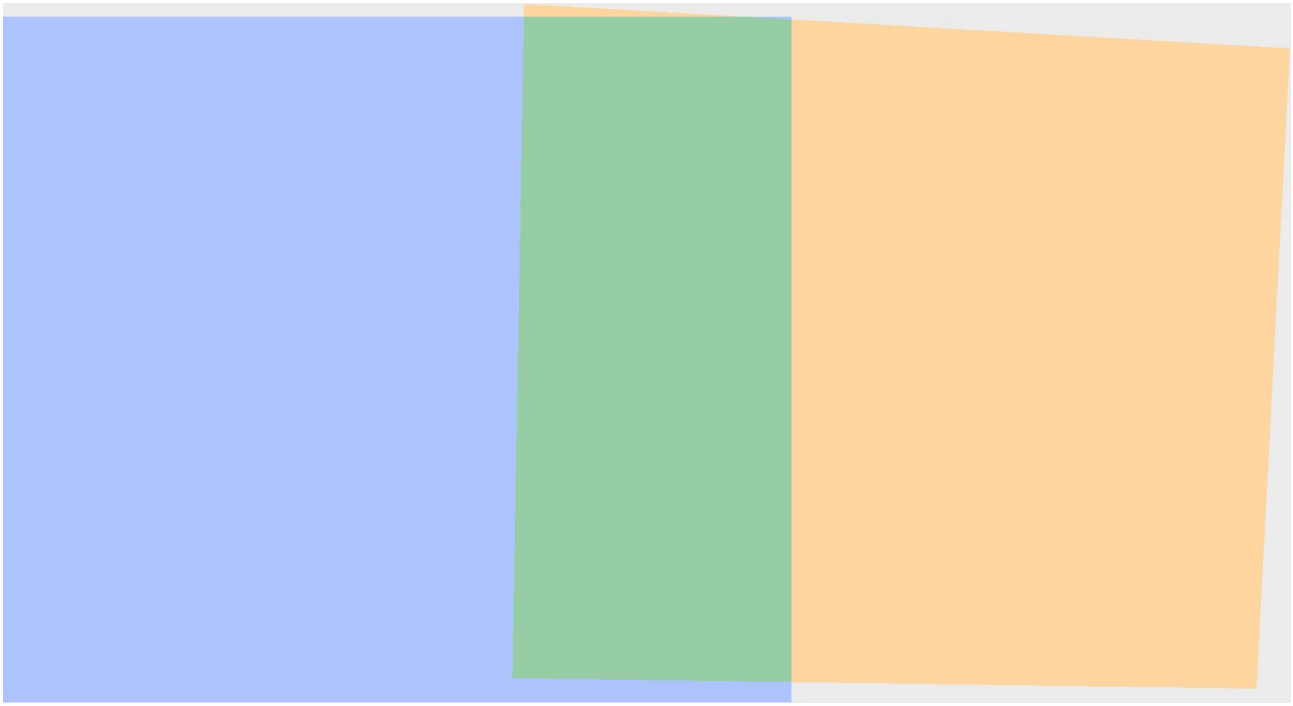


Figure 5. The four regions of the canvas for the example: image 1 only (blue, 762,999 px), image 2 only (orange, 670,151 px), both (green, 384,852 px) and neither (grey, 96,538 px). Each canvas pixel is in exactly one region.

These masks are what a compositing step consumes: the overlap is where a blend is possible, the single-image regions are copied, and the empty corners stay transparent or get a background value. Because NaN propagates through arithmetic, a naive sum of the two canvases is NaN everywhere except the overlap; the companion document shows the usual masked average.

8. Worked example, end to end

The pair is the synthetic one from the companion document: image A is a 1150×1000 crop of test/img1.png and image B a 1150×1000 view of its right part through a known rotation, scale and perspective. cusift_estimate_homography with seed 42 returns the same matrix on every run (the estimate is bit-identical to the earlier document's). Estimated matrix, its inverse, and the ground truth:

$$\mathbf{H} = \begin{vmatrix} 0.94709 & 0.01663 & -718.55048 \\ -0.05530 & 0.95457 & 60.16744 \\ -4.145e\text{-}05 & -3.080e\text{-}05 & 1.00000 \end{vmatrix} \quad \mathbf{H}^{-1} = \begin{vmatrix} 1.09051 & 0.00627 & 783.20750 \\ 0.06021 & 1.04591 & -19.66402 \\ 4.706e\text{-}05 & 3.247e\text{-}05 & 1.03186 \end{vmatrix}$$

$$\mathbf{H}_{\text{true}} = \begin{vmatrix} 0.95143 & 0.01642 & -721.79091 \\ -0.05294 & 0.95795 & 58.07634 \\ -3.791e\text{-}05 & -3.056e\text{-}05 & 1.00000 \end{vmatrix} \qquad \det \mathbf{H} = 0.87705$$

8.1 Canvas

Corners of B transported through $\mathbf{H}^{-1}$ (table in section 4.1): q_0 = (759.0, -19.1), q_1 = (1875.1, 45.6), q_2 = (1826.3, 978.6), q_3 = (741.8, 963.3). Bounding box with image A: u ∈ [0, 1876], v ∈ [-20, 999]; canvas 1877 × 1020; origin (0, -20). The library's cusift_warp returned exactly these dimensions.

8.2 Four canvas pixels traced by hand

Each trace follows section 5.2 literally and compares the hand result with the kernel's output at that pixel.

(a) in the overlap

canvas pixel (x, y)	(1000, 520)
reference point (u, v) = (x + u0, y + v0)	(1000, 500)
inside [0, w1−1) × [0, h1−1)?	yes
C1 value: I1(u, v) by direct lookup	61.000
z = h20·u + h21·v + h22	0.943152
(sx, sy) = numerators / z	(251.1346, 511.2117)
inside [0, w2−1) × [0, h2−1)?	yes
x0, y0, fx, fy	251, 511, 0.1346, 0.2117
neighbours I00, I10, I01, I11	76, 32, 75, 33
weights	0.6822, 0.1061, 0.1832, 0.0285
C2 value, double precision	69.9221
C2 value from the GPU kernel (float32)	69.9230

(b) image A only

canvas pixel (x, y)	(300, 300)
reference point (u, v) = (x + u0, y + v0)	(300, 280)
inside [0, w1−1) × [0, h1−1)?	yes
C1 value: I1(u, v) by direct lookup	232.000
z = h20·u + h21·v + h22	0.978942
(sx, sy) = numerators / z	(-439.0102, 317.5434)
inside [0, w2−1) × [0, h2−1)?	no
C2 value	NaN (kernel also NaN)

(c) image B only

canvas pixel (x, y)	(1500, 520)
reference point (u, v) = (x + u0, y + v0)	(1500, 500)
inside [0, w1−1) × [0, h1−1)?	no
C1 value: I1(u, v) by direct lookup	NaN
z = h20·u + h21·v + h22	0.922427
(sx, sy) = numerators / z	(770.1467, 492.7208)
inside [0, w2−1) × [0, h2−1)?	yes
x0, y0, fx, fy	770, 492, 0.1467, 0.7208
neighbours I00, I10, I01, I11	39, 40, 41, 40
weights	0.2383, 0.0410, 0.6150, 0.1058
C2 value, double precision	40.3767
C2 value from the GPU kernel (float32)	40.3767

(d) neither image

canvas pixel (x, y)	(1850, 5)
reference point (u, v) = (x + u0, y + v0)	(1850, -15)
inside [0, w1−1) × [0, h1−1)?	no
C1 value: I1(u, v) by direct lookup	NaN
z = h20·u + h21·v + h22	0.923779
(sx, sy) = numerators / z	(1118.5792, -61.1200)
inside [0, w2−1) × [0, h2−1)?	no
C2 value	NaN (kernel also NaN)

In trace (a) the double-precision hand value and the float32 kernel value differ by 0.0009 on a 0 to 255 scale, which is float32 rounding in the coordinate chain, not a difference in the method.

8.3 The NumPy re-implementation against the kernel

Appendix A lists a vectorised NumPy version of the canvas construction and of the kernel, written in float32 to mirror the device arithmetic. Run on the same inputs and compared with the library's two canvases:

Quantity	Result
validity mask of canvas 1: pixels that disagree	0
validity mask of canvas 2: pixels that disagree	0
canvas 1: maximum absolute difference	0 (bit-identical)
canvas 2: maximum absolute difference	0.0231 of 255
canvas 2: mean absolute difference	1.66e-04 of 255
source coordinate, float32 versus float64: maximum error	3.00e-04 px

The masks agree pixel for pixel, so the two implementations draw the same boundary, and the first canvas is identical. The residual on the second canvas is the signature of the GPU compiler fusing multiply-add pairs (which changes rounding) and of a different evaluation order in the division; a coordinate error of a few 10^{-4} px multiplied by a local image gradient of up to a hundred intensity levels per pixel gives differences of a few hundredths, as observed.

9. Numerical remarks

- **float32 is enough for the coordinates.** Canvas coordinates are a few thousand at most, where a float32 unit in the last place is about $2^{-13} \approx 1.2 \cdot 10^{-4}$ px. z is near 1 and the perspective products $h_{20}u$ are of order 0.05, so there is no cancellation in the numerators or the denominator. Only for translations in the tens of thousands of pixels would sub-pixel precision begin to erode.
- **The division is the only non-linear step**, one per pixel. On the GPU it is a true IEEE division (the kernel does not use the fast reciprocal intrinsics), so the result is correctly rounded.
- **NaN is a deliberate value, not an accident.** It marks absence of data in both canvases, propagates through any arithmetic the caller does, and tests false in every comparison, which is what makes the range test safe even when z misbehaves.
- **The determinant threshold** of 10^{-12} is far below any matrix a real estimate produces; it exists to stop an all-zero or rank-deficient H from reaching the corner projection, where the division by z would be the first symptom.
- **Resampling quality** is bilinear with no pre-filtering. It is exact for image 1, mild for image 2 near unit scale, and aliasing-prone when |det J| grows well beyond 1. If a stitch must cross a large scale change, downsample the larger image before extraction rather than relying on the warp to do it.

10. The mapping chain in one table

Step	Formula	Who	Output
canvas pixel to reference point	$(u, v) = (x + u_0, y + v_0)$	T^{-1}, integer shift	lattice point of image 1's frame
first canvas	$C_1 = I_1(u, v)$ if inside, else NaN	direct lookup	exact copy, no interpolation
reference point to image 2	$z = h_{20}u + h_{21}v + h_{22}$; $(s_x, s_y) = (N_x, N_y) / z$	H, then π	real-valued position in image 2
second canvas	$C_2 = \text{bilinear}(I_2; s_x, s_y)$ if inside, else NaN	4 neighbours, 4 weights	resampled image 2
canvas size (once)	$q_k = \pi(H^{-1}c_k)$; floor/ceil of the box with image 1	ComputeCanvasGeometry; public: cusift_compute_canvas	outW, outH, (u_0, v_0), corners

Appendix A. NumPy reference implementation

This is the code that produced the comparison of section 8.3. It uses float32 where the kernel does, truncation for the floor as the kernel does, and the same half-open range test. It is a specification in executable form, not a replacement for the kernel: on the 1877×1020 example it is about two orders of magnitude slower than the GPU.

```
import numpy as np

def canvas_geometry(H, w1, h1, w2, h2):
    """Replicates cusift_warp steps 1-2: corners of image 2 through inv(H), then the box."""
    Hinv = np.linalg.inv(H.astype(np.float64))
    c = np.array([[0, 0, 1], [w2-1, 0, 1], [w2-1, h2-1, 1], [0, h2-1, 1]], float)
    q = (Hinv @ c.T).T;  q = q[:, :2] / q[:, 2:3]               # perspective divide
    uMin, uMax = min(0.0, q[:, 0].min()), max(w1-1.0, q[:, 0].max())
    vMin, vMax = min(0.0, q[:, 1].min()), max(h1-1.0, q[:, 1].max())
    u0, u1 = int(np.floor(uMin)), int(np.ceil(uMax))
    v0, v1 = int(np.floor(vMin)), int(np.ceil(vMax))
    return u0, v0, u1-u0+1, v1-v0+1, q

def bilinear(img, sx, sy):
    """Half-open domain [0, w-1) x [0, h-1); NaN outside.  Same arithmetic as WarpDualKernel."""
    h, w = img.shape
    valid = (sx >= 0) & (sx < w-1) & (sy >= 0) & (sy < h-1)
    sx = np.where(valid, sx, 0).astype(np.float32); sy = np.where(valid, sy, 0).astype(np.float32)
    x0 = sx.astype(np.int32); y0 = sy.astype(np.int32)             # truncation == floor for sx >= 0
    fx = sx - x0; fy = sy - y0
    val = ((1-fx)*(1-fy)*img[y0, x0] + fx*(1-fy)*img[y0, x0+1]
           + (1-fx)*fy*img[y0+1, x0] + fx*fy*img[y0+1, x0+1])
    return np.where(valid, val, np.nan).astype(np.float32)

def warp_pair(img1, img2, H):
    h1, w1 = img1.shape; h2, w2 = img2.shape
    u0, v0, outW, outH, _ = canvas_geometry(H, w1, h1, w2, h2)
    Hf = H.astype(np.float32)
    U = (np.arange(outW, dtype=np.float32) + np.float32(u0))[None, :].repeat(outH, 0)   # u = x + u0
    V = (np.arange(outH, dtype=np.float32) + np.float32(v0))[:, None].repeat(outW, 1)   # v = y + v0
    C1 = bilinear(img1, U, V)                                       # integer (u, v): exact copy
    z  = Hf[2, 0]*U + Hf[2, 1]*V + Hf[2, 2]
    sx = (Hf[0, 0]*U + Hf[0, 1]*V + Hf[0, 2]) / z
    sy = (Hf[1, 0]*U + Hf[1, 1]*V + Hf[1, 2]) / z
    C2 = bilinear(img2, sx, sy)
    return C1, C2, (u0, v0)
```

Appendix B. The example's numbers at a glance

Quantity	Value
image sizes	A: 1150 × 1000; B: 1150 × 1000
RANSAC inliers (the reported inlier count)	871
det H	0.87705
corner z values through H^{-1}	1.0319, 1.0859, 1.1184, 1.0643
transported corners q0 … q3	(759.0, -19.1); (1875.1, 45.6); (1826.3, 978.6); (741.8, 963.3)
bounding box	u in [0, 1876], v in [-20, 999]
canvas, origin	1877 × 1020, (0, -20)
footprint area (shoelace), overlap polygon area	1,054,998 px², 385,179 px²
mask counts: only A / only B / both / none	762,999 / 670,151 / 384,852 / 96,538
det J over the footprint	0.964 to 1.227; lattice sum 1,147,856 vs (w2−1)(h2−1) = 1,147,851
distance of the exceptional lines from the images	about 19 000 px (reference frame), 18 000 px (image 2)
midpoint of the top edge of B, as a fraction of the transported edge	0.5128